

Rulmanlarda Erken Arıza Uyarısı

NASA/IMS arızaya kadar çalıştırma (run-to-failure) rulman veri setinde gözetimsiz anomali tespiti ve erken uyarı

Teknik Rapor

Ecem Aydemir

Ekim 2026

Özet

Yuvarlanmalı rulmanlar, döner makinelerdeki arızaların en yaygın kaynaklarından biridir ve uygulamada arıza etiketleri neredeyse hiçbir zaman bulunmaz. Bu rapor, bir makinenin ömrünün ilk kısmından *sağlıklı* titreşimin nasıl görüldüğünü öğrenen ve bir rulman bu durumdan uzaklaştığında kararlı (debounce edilmiş) bir alarm üreten, uçtan uca ve yeniden üretilebilir bir erken uyarı sistemini anlatmaktadır. Sistem, 20 kHz'de örneklenmiş her bir saniyelik titreşim kaydını fiziksel anlamı olan 18 durum göstergesine (zaman alanı istatistikleri, frekans bandı enerjileri ve rulman arıza frekanslarındaki zarf spektrumu enerjisi) indirger; dört dedektörü (RMS eşiği temel modeli, Mahalanobis uzaklığı, Isolation Forest ve LSTM otokodlayıcı) karşılaştırır ve zarf spektrumuna dayalı bir arıza tipi teşhisi ekler. Geliştirme testinde (IMS test 2) tüm dedektörler, arızalanan rulmanda testin bitiminden **yaklaşık 74 saat önce (deney süresinin %45'i)** kalıcı bir alarm verir ve arızayı doğru biçimde **dış bilezik** hasarı olarak teşhis eder. Ayrılmış (held-out) iki testte ise sonuçlar, uyarı süresi ile yanlış alarm sayısı arasındaki açık bir ödünleşimi ve dürüst bir sınırlamayı ortaya koyar: makine duruşlarından sonra değişen çalışma koşulları anomali gibi görünmektedir. Çalışma; tek komutla çalışan bir veri işleme hattı, birim testleri, açıklamalı bir notebook ve etkileşimli bir Streamlit paneli içeren açık kaynaklı bir Python paketi olarak sunulmaktadır.

İçindekiler

1	Giriş	3
1.1	Problem	3
1.2	Tek paragrafta yaklaşım	3
1.3	Katkılar	3
1.4	Önceki bir denemeye ilişkisi	3
2	Teorik Arka Plan	4
2.1	Yuvarlanmalı rulmanlar ve hasar tipleri	4
2.2	Titreşim durum göstergeleri	5
2.3	Gözetimsiz anomali tespiti	5
3	Veri Seti	6
4	Kullanılan Araçlar ve Ortam	7

5	Yöntem	7
5.1	Öznitelik çıkarımı	8
5.2	Sinyal düzeyinde örnekleme	9
5.3	Ön işleme ve veri bölmeleri	10
5.4	Dedektörler	11
5.5	Puandan alarma	12
5.6	Teshis	13
5.7	Değerlendirme protokolü	13
5.8	Tasarım kararları	13
6	Sonuçlar	14
6.1	Ana sonuçlar	14
6.2	Test 2 (geliştirme kümesi)	16
6.3	Test 1 (ayrılmış)	18
6.4	Test 3 (ayrılmış)	20
6.5	Bulguların özeti	21
7	Etkileşimli Panel (Dashboard)	22
8	Yazılım Mühendisliği ve Yeniden Üretilebilirlik	24
8.1	Depo yapısı	24
8.2	Yapılandırma	24
8.3	Çalıştırma	24
8.4	Testler	24
8.5	Sürüm kontrolü	25
9	Tartışma ve Sınırlamalar	25
10	Sonuç ve Gelecek Çalışmalar	25

1 Giriş

1.1 Problem

Kestirimci bakımın amacı bir makineyi arızalanmadan *kısa bir süre önce* onarmaktır: bileşenin ömrünün büyük kısmını kullanacak kadar geç, müdahaleyi planlayacak kadar erken. Döner makineler için en zengin bilgi kaynağı titreşimdir ve en sık arızalanan bileşen rulmanlardır. Sensörlü bir makine sürekli titreşim verisi üretir; ancak bu veri neredeyse hiçbir zaman “bu rulman şu an bozulmaya başladı” diyen etiketlerle gelmez. Arızalar nadir, pahalı ve birbirinden farklıdır; bu nedenle bilinen arızaları sınıflandırmak üzere eğitilmiş bir model yeni bir makine için genellikle mevcut değildir.

Bu projenin cevapladığı soru şudur:

Yalnızca makinenin sağlıklı olduğu varsayılan bir dönemin verisini kullanarak, bir operatörü bir rulmanın arızalanmaya başladığı konusunda erken, güvenilir ve az sayıda yanlış alarmla uyabilir ve arızanın muhtemel tipini söyleyebilir miyiz?

1.2 Tek paragrafta yaklaşım

Her titreşim kaydı kompakt bir durum göstergesi vektörüne dönüştürülür. Her deneyin ilk %25’lik kısmı sağlıklı kabul edilir ve bir anomali dedektörünü eğitmek için kullanılır; sonraki %10’luk kısım alarm eşiğini belirler. Bu noktadan sonra dedektör her yeni kaydı puanlar; son 6 puanın en az 4’ü eşiği aştığında alarm verilir ve zarf enerjisi en çok artan arıza frekansı muhtemel arıza tipini belirtir. Sistem üç adet arızaya kadar çalıştırma deneyinde, alarm ile testin bitişi arasındaki saat sayısı (*uyarı süresi*, lead time) ve makine henüz sağlıklıken verilen alarm sayısı (*yanlış alarm*) ile değerlendirilmiştir.

1.3 Katkıları

- IMS veri seti için sızıntısız (leakage-free), zaman sıralı bir değerlendirme protokolü; tüm üç teste ve arızalanan tüm rulmanlara uygulanmıştır. Test 2 geliştirme için kullanılmış, test 1 ve test 3 ayrılmış (held-out) olarak tutulmuştur.
- Test düzeneğindeki rulmanın gerçek geometrisi için hesaplanmış, arıza frekanslarında zarf analizi içeren fiziksel temelli öznitelikler.
- Klasik bir temel model, iki klasik makine öğrenmesi dedektörü ve bir derin öğrenme dizi modelinin aynı veri bölmeleri, eşikler ve alarm mantığı altında karşılaştırılması.
- Operatöre yönelik alarm mantığı (debounce, kalıcılık, ilk alarm veren rulmanla kaynak tespiti ve arıza teşhisi) ve bir testi canlı veri akışı gibi yeniden oynatan etkileşimli bir panel.
- Yeniden üretilebilir bir yazılım paketi: yapılandırma dosyası, Makefile, birim testleri ve depoya eklenmiş öznitelik tabloları (3 MB). Böylece tüm sonuçlar 6 GB ham veriyi indirmeden yeniden üretilebilir.

1.4 Önceki bir denemeyle ilişkisi

Bu proje, aynı veri setinde bir LSTM otokodlayıcı eğiten daha önceki bir notebook’un (2025) yeniden ele alınmasıdır. O çalışmanın incelenmesi, sonuçlarını geçersiz kılan birkaç yöntemsel sorunu ortaya çıkardı. Bu sorunlar bu veri setinde sık yapılan hatalar olduğu ve Bölüm 5.8’deki tasarım kararlarını doğrudan yönlendirdiği için burada listelenmiştir:

1. **Model arızalı veriyle eğitilmişti.** Eğitim kümesi deneyin *son* bir milyon örneği idi; yani rulmanın bozulduğu dönem. Aynı veri daha sonra test için de kullanılmıştı.
2. **Yanlış rulman izleniyordu.** Test 1'in yalnızca ilk kanalı (rulman 1) kullanılmıştı; oysa test 1'de arızalanan rulmanlar 3 ve 4'tür, rulman 1 sağlam kalır.
3. **Zaman bilgisi kaybolmuştu.** 2156 dosyanın tamamı uç uca eklenip 50 örneklik (2,5 ms) pencerelere bölünmüştü. Zaman eksenini olmadan "arızadan kaç saat önce uyardı?" sorusu cevaplanamaz.
4. **Eğitim ve doğrulama arasında sızıntı.** Üst üste binen pencereler eğitim/doğrulama bölmesinden önce karıştırılmıştı; bu nedenle doğrulama kaybı olduğundan iyi görünüyordu.

Önceki çalışmanın temel fikri ("normal"i bir otokodlayıcıyla öğrenip yeniden oluşturma hatasını anomali puanı olarak kullanmak) korunmuştur; bu rapordaki LSTM otokodlayıcı dedektörü, aynı fikrin doğru kurulmuş hâlidir.

2 Teorik Arka Plan

2.1 Yuvarlanmalı rulmanlar ve hasar tipleri

Bir yuvarlanmalı rulman; mile takılı iç bilezik, gövdeye takılı dış bilezik, yuvarlanma elemanları (bilye veya makaralar) ve bu elemanları birbirinden ayıran kafesten oluşur. Yorulma, kirlenme veya yetersiz yağlama bu yüzeylerden birinde lokal bir hasar (oyuk ya da çatlak) oluşturur. Bir yuvarlanma elemanı hasarın üzerinden her geçtiğinde kısa bir mekanik darbe oluşur. Bu darbeler yalnızca rulman geometrisine ve mil hızına bağlı bir frekansla tekrarlanır ve rulman ile gövdenin, genellikle kHz mertebesindeki yapısal rezonanslarını uyarır.

Mil frekansı f_r , n adet çapı d olan yuvarlanma elemanı, hatve çapı D ve temas açısı ϕ için karakteristik arıza frekansları şunlardır:

$$\text{BPFO} = \frac{n}{2} f_r \left(1 - \frac{d}{D} \cos \phi \right) \quad (\text{dış bilezik geçiş frekansı}) \quad (1)$$

$$\text{BPFI} = \frac{n}{2} f_r \left(1 + \frac{d}{D} \cos \phi \right) \quad (\text{iç bilezik geçiş frekansı}) \quad (2)$$

$$\text{BSF} = \frac{D}{2d} f_r \left(1 - \left(\frac{d}{D} \cos \phi \right)^2 \right) \quad (\text{yuvarlanma elemanı dönme frekansı}) \quad (3)$$

$$\text{FTF} = \frac{f_r}{2} \left(1 - \frac{d}{D} \cos \phi \right) \quad (\text{kafes frekansı}) \quad (4)$$

IMS test düzeneği, sıra başına $n = 16$ makaralı, $d = 0,331$ inç, $D = 2,815$ inç ve $\phi = 15,17^\circ$ olan Rexnord ZA-2115 çift sıralı rulmanlar kullanır ve 2000 dev/dk ($f_r = 33,3$ Hz) hızla döner. Elde edilen frekanslar Tablo 1'de, kod karşılığı Kod 1'de verilmiştir.

Tablo 1: ZA-2115 rulmanının 2000 dev/dk'daki karakteristik arıza frekansları.

Frekans	Değer [Hz]	İşaret ettiği hasar
BPFO	236,4	dış bilezik
BPFI	296,9	iç bilezik
BSF	139,9	yuvarlanma elemanı
FTF	14,8	kafes

Kod 1: Rulman geometrisinden arıza frekansları (src/bearing_efw/dataset.py).

```

1 FS = 20_480 # sampling rate [Hz]
2 SHAFT_HZ = 2000 / 60.0
3 N_ROLLERS, ROLLER_D, PITCH_D, CONTACT_ANGLE_DEG = 16, 0.331, 2.815, 15.17
4
5 def fault_frequencies(fr: float = SHAFT_HZ) -> dict[str, float]:
6     """Characteristic defect frequencies [Hz] for the ZA-2115 bearing."""
7     ratio = ROLLER_D / PITCH_D * np.cos(np.deg2rad(CONTACT_ANGLE_DEG))
8     return {
9         "bpfo": N_ROLLERS / 2 * fr * (1 - ratio), # outer race
10        "bpfi": N_ROLLERS / 2 * fr * (1 + ratio), # inner race
11        "bsf": PITCH_D / (2 * ROLLER_D) * fr * (1 - ratio**2), # rolling element
12        "ftf": fr / 2 * (1 - ratio), # cage
13    }

```

2.2 Titreşim durum göstergeleri

Zaman alanı istatistikleri. Ortalaması sıfıra çekilmiş bir $x_1, \dots, x_N$ kaydı için:

$$\text{RMS} = \sqrt{\frac{1}{N} \sum_i x_i^2}, \quad \text{basıklık (kurtosis)} = \frac{\frac{1}{N} \sum_i x_i^4}{\text{RMS}^4}, \quad \text{tepe faktörü} = \frac{\max_i |x_i|}{\text{RMS}}. \quad (5)$$

RMS toplam titreşim enerjisini ölçer ve endüstriyel standartların çoğunun (ör. ISO 10816/20816) kullandığı göstergedir. Basıklık Gauss gürültüsü için 3'tür ve sinyal keskin darbeler içerdiğinde büyür; bu nedenle enerji artmadan önce bile erken, lokal hasarlara duyarlıdır. Tepe (crest), biçim ($\text{RMS}/|x|$) ve darbe ($\max |x|/|x|$) faktörleri benzer darbesellik ölçüleridir; çarpıklık (skewness) asimetriyi yakalar.

Spektral bant enerjileri. Güç spektrumunun altı banttaki (0–1, 1–2, 2–4, 4–6, 6–8 ve 8–10,24 kHz) payı ile spektral ağırlık merkezi, enerjinin *nerede* toplandığını tanımlar. Gelişen bir hasar enerjisi yüksek frekanslı rezonanslara doğru kaydırır.

Zarf (envelope) analizi. Hasar darbeleri kısa sürelidir ve yüksek frekanslı rezonansları uyarır; bunların *tekrarlanma hızı* (ör. BPFO) rezonansın altında gizli kaldığı için ham spektrumda görünmez. Zarf analizi bu bilgiyi üç adımda geri kazanır [3]:

1. sinyal rezonans bölgesinde bant geçiren filtreden geçirilir (burada 2–8 kHz, 4. dereceden Butterworth, sıfır faz),
2. analitik sinyalin (Hilbert dönüşümü) genliği alınır; bu, genlik modülasyonlu rezonansın zarfıdır,
3. zarfın spektrumu hesaplanır; lokal bir hasar, karakteristik frekansında ve harmoniklerinde tepe üretir.

Burada kullanılan öznitelik, zarf spektrumu enerjisinin (5–1000 Hz) BPFO, BPFI ve BSF'nin ilk üç harmoniğinin ± 3 Hz çevresine düşen payıdır.

2.3 Gözetimsiz anomali tespiti

Arıza etiketi kullanılmadığı için tüm dedektörler *tek sınıflı* (one-class) veya *yenilik* (novelty) dedektörleridir: yalnızca sağlıklı veriyle eğitilirler ve yeni bir gözlem sağlıklı veriye ne kadar az benziyorsa o kadar büyük bir puan üretirler.

Mahalanobis uzaklığı. Sağlıklı öznitelik vektörlerinin ortalaması μ ve kovaryansı Σ ile

$$d_M(\mathbf{x}) = \sqrt{(\mathbf{x} - \mu)^\top \Sigma^{-1} (\mathbf{x} - \mu)}. \quad (6)$$

Bu ölçü öznitelikler arasındaki korelasyonları (ör. RMS ve tepe değeri) hesaba katar. Pek çok öznitelik birbiriyle ilişkili ve eğitim kümesi küçük olduğundan Σ , örnek kovaryansı ölçeklenmiş birim matrise doğru çekerek iyi koşullu tutan Ledoit–Wolf büzülme (shrinkage) yöntemiyle kestirilir [5].

Isolation Forest. Gözlemleri rastgele bölmelerle yalıtın rastgele ağaçlardan oluşan bir topluluktur [4]. Anomaliler az sayıda ve farklı oldukları için daha az bölmeyle yalıtılır; puan ortalama yol uzunluğundan türetilir.

LSTM otokodlayıcı. Otokodlayıcı girdisini düşük boyutlu bir gizli koda sıkıştırır ve yeniden oluşturur. Yalnızca sağlıklı veriyle eğitildiğinde sağlıklı girdileri iyi, tanımadığı girdileri kötü yeniden oluşturur; bu nedenle W kayıt ve p öznitelikten oluşan bir pencere üzerindeki yeniden oluşturma hatası

$$e_t = \frac{1}{Wp} \sum_{k=0}^{W-1} \sum_{j=1}^p (x_{t-k,j} - \hat{x}_{t-k,j})^2 \quad (7)$$

bir anomali puanıdır [7]. Uzun kısa süreli bellek (LSTM) katmanları [6] modelin pencere içindeki zamansal değişimi kullanmasını sağlar.

RMS eşiği. Her bakım mühendisinin ilk deneyeceği temel yöntem: toplam titreşim seviyesi sağlıklı aralığının üstüne çıktığında alarm. Daha karmaşık bir yöntemin karmaşıklığına değmesi için bunu geçmesi gerekir.

3 Veri Seti

IMS rulman veri seti [1, 2], Cincinnati Üniversitesi’ndeki NSF I/UCR Akıllı Bakım Sistemleri Merkezi (Center for Intelligent Maintenance Systems) tarafından üretilmiş ve NASA Prognostik Mükemmeliyet Merkezi (PCoE) tarafından dağıtılmaktadır. Bir AC motorla sabit 2000 dev/dk’da döndürülen tek bir mil üzerinde dört rulman bulunur ve bir yay mekanizmasıyla 6000 lb radyal yük uygulanır. Rulman gövdelerindeki ivmeölçerler (PCB 353B33) bir NI DAQCard-6062E ile 20 kHz’de örneklenir. Her 10 dakikada bir (test 1’in başında 5 dakikada bir) kanal başına 20 480 örnekten oluşan bir saniyelik bir kayıt, adı zaman damgası olan bir metin dosyasına yazılır. Her test bir rulman arızalanana kadar sürdürülmüştür.

Tablo 2: Üç arızaya kadar çalıştırma testi. Süre, ölü kayıtlar çıkarıldıktan sonra ölçülmüştür.

Test	Kayıt	Kanal	Süre [sa]	Test sonundaki arıza
test1	2 156	8 (rulman başına x, y)	827,6	rulman 3 iç bilezik, rulman 4 yuvarlanma elemanı
test2	984	4	163,5	rulman 1 dış bilezik
test3	6 324	4	1 073,1	rulman 3 dış bilezik

Veriyle çalışırken keşfedilen pratik ayrıntılar.

- Arşiv, içinde bir 7-Zip dosyası, onun içinde de üç RAR dosyası bulunan bir ZIP’tir (yaklaşık 1 GB indirme, açıldığında 6,2 GB, 9 464 kayıt dosyası).

- `3rd_test.rar` dosyası `4th_test/txt` adlı bir klasöre açılır.
- Dosyalar sekmeyeyle ayrılmış, Windows satır sonlu ve başlıksızdır.
- Test 2 ve test 3'ün son kaydı, kapanıştan sonra kaydedilmiş sifıra yakın bir sinyaldir (RMS $\approx 0,001$ g). Bu “ölü” kayıtlar çıkarılır (herhangi bir kanalda RMS $< 0,01$ g).
- Test 1, 148 saate varan boşluklarla parçalar hâlinde kaydedilmiştir; test 3'te 4,5, 7,5 ve 7,3 saatlik duruşlar vardır (yaklaşık 262., 354. ve 1002. saatlerde). Test 2 kesintisizdir.

İndirme. `scripts/download_data.py`, arşivi NASA deposunun PHM Society aynasından indirir ve tüm katmanları açar (`py7zr` ile `unar` veya `unrar` komutu gerekir). Ham veri Git'e eklenmez; çıkarılmış öznitelikler depoya eklenmiştir.

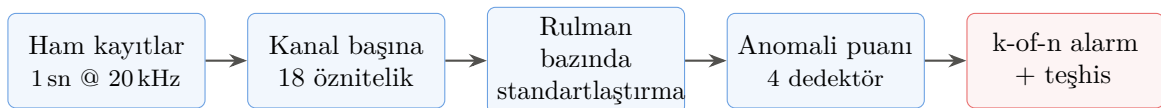
4 Kullanılan Araçlar ve Ortam

Tablo 3: Projede kullanılan yazılımlar.

Araç	Görevi
Python 3.10+	uygulama dili
NumPy, pandas	sayısal diziler, öznitelik tabloları, zaman işlemleri
SciPy (<code>signal</code> , <code>stats</code>)	Butterworth filtreleme, Hilbert dönüşümü, basıklık/çarpıklık
scikit-learn	Ledoit–Wolf kovaryansı, Isolation Forest
PyTorch	LSTM otokodlayıcı (CPU yeterlidir)
Matplotlib	README ve bu rapor için statik grafikler
Streamlit, Plotly	etkileşimli yeniden oynatma paneli
PyYAML	deney yapılandırması (<code>configs/default.yaml</code>)
pytest	birim testleri ve panel duman testleri (Streamlit <code>AppTest</code>)
Jupyter / nbconvert	çalıştırılmış açıklamalı notebook
py7zr, unar	iç içe 7z/RAR arşivinin açılması
GNU Make	tek komutluk giriş noktaları (<code>make experiment</code> , <code>make dashboard</code> , ...)
Git, GitHub	sürüm kontrolü, pull request'ler
L ^A T _E X (pdfLaTeX, TikZ, listings)	bu rapor

Tüm hat bir dizüstü bilgisayar işlemcisinde çalışır: 9 464 dosyadan öznitelik çıkarımı 4 süreçle yaklaşık 1,5 dakika, dört dedektörün üç testte eğitilip değerlendirilmesi yaklaşık 1,5 dakika sürer.

5 Yöntem



9 464 dosya (6 GB) → 3 MB öznitelik tablosu → her rulman ve kayıt için bir puan ve bir alarm durumu

Şekil 1: Veri işleme hattı.

5.1 Öznitelik çıkarımı

Her kayıt ve kanal 18 öznitelığe indirgenir (Kod 2): 8 zaman alanı istatistiği, 6 görelî bant enerjisi, spektral ağırlık merkezi ve 3 zarf enerjisi. Test 1’de rulman başına iki kanal olduğundan test 1 rulmanları 36, test 2 ve test 3 rulmanları 18 öznitelikle tanımlanır.

Kod 2: Spektral ve zarf öznitelikleri (src/bearing_efw/features.py’den alıntı).

```

1 BANDS = [(0, 1000), (1000, 2000), (2000, 4000), (4000, 6000), (6000, 8000), (8000, 10240)]
2 ENVELOPE_BAND = (2000, 8000)
3 FAULT_TOL_HZ = 3.0 # half-width of the window around each defect frequency
4 N_HARMONICS = 3
5 _SOS = signal.butter(4, ENVELOPE_BAND, btype="bandpass", fs=FS, output="sos")
6
7 def spectral_features(x: np.ndarray) -> dict[str, float]:
8     x = x - x.mean()
9     freqs = np.fft.rfftfreq(len(x), 1 / FS)
10    psd = np.abs(np.fft.rfft(x * np.hanning(len(x)))) ** 2 / len(x)
11    total = psd.sum()
12    out = {f"band_{lo // 1000}_{hi // 1000}k": _band_energy(freqs, psd, lo, hi) / total
13           for lo, hi in BANDS}
14    out["spectral_centroid"] = float((freqs * psd).sum() / total)
15
16    # Envelope spectrum: impacts from a localized defect modulate the resonance
17    # at the defect frequency, which shows up after demodulation.
18    env = np.abs(signal.hilbert(signal.sosfiltfilt(_SOS, x)))
19    env -= env.mean()
20    env_spec = np.abs(np.fft.rfft(env * np.hanning(len(env)))) / len(env)
21    env_total = env_spec[(freqs > 5) & (freqs < 1000)].sum()
22    for name in ("bpfo", "bpfi", "bsf"):
23        e = sum(_band_energy(freqs, env_spec,
24                             k * _FAULT_F[name] - FAULT_TOL_HZ, k * _FAULT_F[name] +
25                             FAULT_TOL_HZ)
26               for k in range(1, N_HARMONICS + 1))
27    out[f"env_{name}"] = e / env_total
28    return out

```

Bant ve zarf enerjilerinin mutlak değil görelî olarak kullanılmasının nedeni, bu özniteliklerin spektrumun *biçimini* tanımlaması ve RMS’in birer kopyası olmamasıdır. Öznitelik çıkarımı bir süreç havuzuyla dosyalar üzerinde paralelleştirilmiştir (Kod 3); sonuç, her test için bir sıkıştırılmış CSV dosyasıdır.

Kod 3: Paralel öznitelik çıkarımı (scripts/build_features.py’den alıntı).

```

1 def _process(args: tuple[str, Path]) -> list[dict]:
2     test, path = args
3     spec = TESTS[test]
4     data = load_snapshot(path) # (20480, n_channels) float32
5     ts = parse_timestamp(path.name) # file name = acquisition time
6     rows = []
7     for bearing, cols in spec.channels.items():
8         for axis, col in enumerate(cols):
9             rows.append({"timestamp": ts, "bearing": bearing, "channel": axis,
10                        **snapshot_features(data[:, col])})
11    return rows
12
13 with ProcessPoolExecutor(workers) as ex:
14     rows = [r for chunk in ex.map(_process, [(test, f) for f in files], chunksize=16)
15            for r in chunk]

```

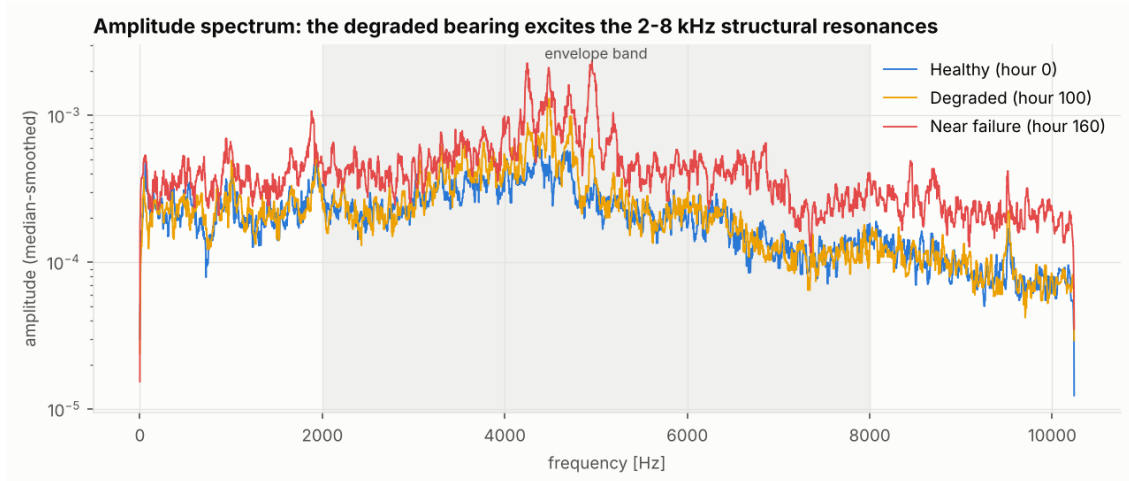
5.2 Sinyal düzeyinde örnekleme

Şekil 2–4, test 2’de arızalanan rulmanda öznelitliklerin neyi yakaladığını üç anda gösterir: sağlıklı (0. saat), bozulmuş (100. saat, alarmdan sonra) ve arızaya yakın (160. saat). Grafik etiketleri depodaki grafiklerle aynı kalması için İngilizcedir.



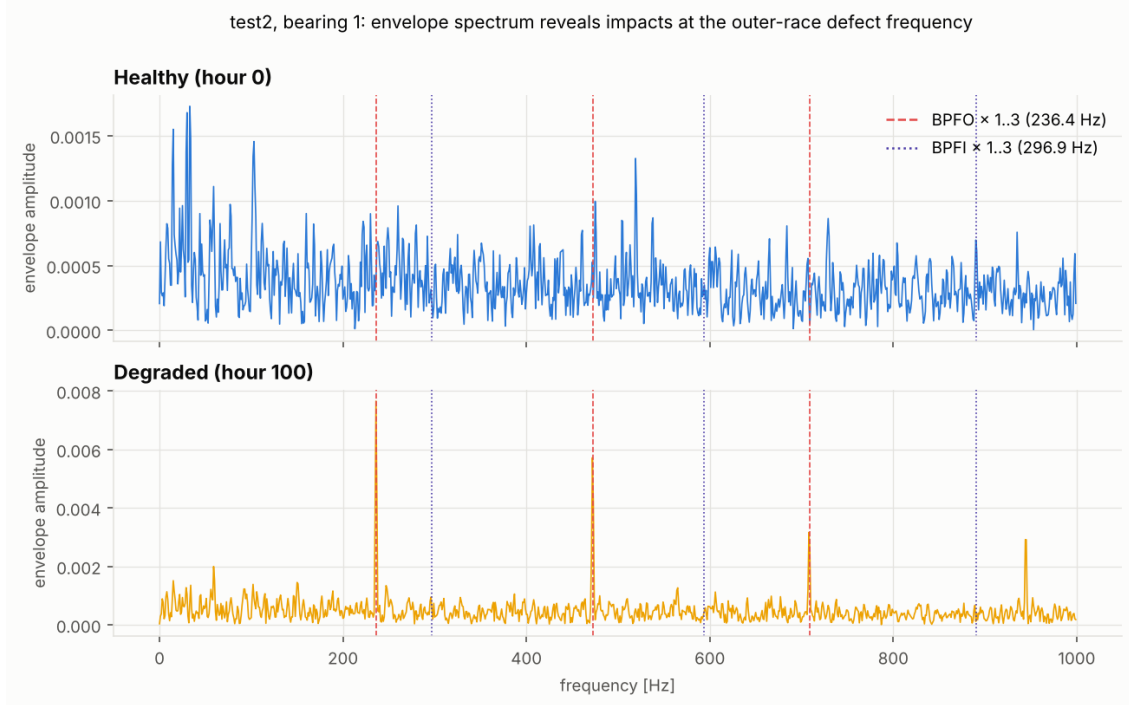
Şekil 2: Test 2, rulman 1’in ham titreşimi (üç kaydın ilk 100 ms’si).

Yorum. 100. saatte dalga biçimi görsel olarak sağlıklı olana hâlâ benzer ve RMS yalnızca 0,073’ten 0,091 g’ye (+%25) çıkmıştır; bu artış gözle veya sabit bir titreşim sınırıyla kolayca gözden kaçabilir. Arızaya yakın dönemde genlik üç katına çıkmış (0,223 g) ve sinyal belirgin biçimde darbeli hâle gelmiştir. Dolayısıyla erken uyarı, dalga biçimine bakmaktan veya toplam seviyeden fazlasına dayanmalıdır.



Şekil 3: Aynı üç kaydın genlik spektrumu (medyan filtreyle yumuşatılmış, logaritmik ölçek).

Yorum. Üç spektrumun da enerjisi yaklaşık 3 ile 6 kHz arasında toplanır: bunlar rulman gövdesinin yapısal rezonanslarıdır. Rulman bozuldukça rezonans tepeleri büyür ve genişler; arızaya yakın dönemde yüksek frekans tabanı (6–10 kHz) neredeyse bir mertebe yükselir. Bu durum hem görelî bant enerjisi özniteliklerini hem de zarf analizi için 2–8 kHz bandının seçimini doğrular. Ancak spektrum, *hangi* parçanın hasarlı olduğunu göstermez.



Şekil 4: Sağlıklı ve bozulmuş kayıtların zarf spektrumu; BPFO’nun (kesikli) ve BPFI’nin (noktalı) ilk üç harmoniği işaretlenmiştir.

Yorum. Bu, teşhis açısından en önemli şekildir. Sağlıklı zarf spektrumu, arıza frekanslarında hiçbir yapı içermeyen düz bir gürültü tabanıdır. 100. saatte tam olarak 236, 473 ve 709 Hz’de, yani BPFO’nun 1×, 2× ve 3× katlarında, gürültü tabanının yaklaşık on katı genlikte keskin tepeler belirir; iç bilezik frekanslarında ise hiçbir şey görünmez. Bu, lokal bir **dış bilezik** hasarının ders kitabı imzasıdır ve testin bitiminden yaklaşık üç gün önce görünürdür. Şekil, rulman geometrisinin, arıza frekansı hesabının ve zarf hattının doğru olduğunu doğrular; `env_bpfo` özniteliğinin ölçtüğü de budur.

5.3 Ön işleme ve veri bölmeleri

Öznitelikler her rulman için bir matrise dönüştürülür (her satır bir kayıt, zaman sırasıyla). Kesin pozitif ve ağır kuyruklu özniteliklerin logaritması alınır; ardından her öznitelik, o rulmanın kendi eğitim döneminin ortalaması ve standart sapmasıyla standartlaştırılır (Kod 4). Bu adımdan sonra +3 değeri, rulman doğası gereği gürültülü ya da sessiz olsun, “bu rulmanın normalinin üç sağlıklı standart sapma üstü” anlamına gelir; böylece bir testteki tüm rulmanlar tek bir dedektörle izlenebilir.

Kod 4: Zaman sıralı bölmeler ve rulman bazında standartlaştırma (`src/bearing_efw/preprocess.py`’den alıntı).

```
1 for bearing, w in wide_by_bearing(df).items():
2     hours = ((w.index - t0).total_seconds() / 3600).to_numpy()
3     # Splits use the snapshot index, not wall-clock time: test1 was recorded in
4     # bursts with multi-day gaps, so a time-based split would leave almost no
5     # training data.
```

```

6   life = np.arange(len(w)) / len(w)
7   train = life < train_end                                # first 25 %
8   calib = (life >= train_end) & (life < calib_end)        # next 10 %
9
10  Z = w.copy()
11  log_cols = [c for c in Z.columns if c.rsplit("_ch", 1)[0] in LOG_FEATURES]
12  Z[log_cols] = np.log(Z[log_cols].clip(lower=1e-9))
13  mu, sd = Z[train].mean(), Z[train].std().replace(0, 1)
14  X = ((Z - mu) / sd).to_numpy(dtype=np.float32)

```

Tablo 4: Rulman başına bölme boyutları.

Test	Kayıt	Eğitim	Kalibrasyon	Eğitim bitişi [sa]	Kalibrasyon bitişi [sa]
test1	2 156	539	216	290,1	433,8
test2	982	246	98	40,8	57,2
test3	6 323	1 581	633	267,5	380,2

5.4 Dedektörler

Tüm dedektörler ortak bir arayüze sahiptir: `fit` dört rulmanın sağlıklı eğitim matrislerinin listesini alır, `score` ise bir rulmanın tüm matrisini kayıt başına bir puana dönüştürür. Tüm rulmanların standartlaştırılmış sağlıklı verisini birleştirerek eğitmek, rulman başına ayrı model eğitmeye göre eğitim kümesini dört katına çıkarır.

Kod 5: Klasik dedektörler (`src/bearing_efw/models.py`'den alıntı).

```

1  class RMSThreshold:
2      """Classic condition-monitoring baseline: overall vibration level (z-score of log RMS)."""
3      def fit(self, Xs): return self
4      def score(self, X): return X[:, self.idx].max(axis=1) # idx = RMS columns
5
6  class Mahalanobis:
7      """Distance from the healthy feature distribution (shrinkage covariance)."""
8      def fit(self, Xs):
9          self.cov = LedoitWolf().fit(np.vstack(Xs))
10         return self
11     def score(self, X):
12         return np.sqrt(self.cov.mahalanobis(X))
13
14  class IForest:
15     def __init__(self, n_estimators=300, seed=0):
16         self.model = IsolationForest(n_estimators=n_estimators, random_state=seed)
17     def fit(self, Xs):
18         self.model.fit(np.vstack(Xs))
19         return self
20     def score(self, X):
21         return -self.model.score_samples(X) # higher = more anomalous

```

LSTM otokodlayıcı mimarisi. Kodlayıcı LSTM (32 gizli birim) ardışık $W = 12$ kayıttan (yaklaşık iki saat) oluşan bir pencereyi okur; son gizli durumu 8 boyutlu bir gizli koda izdüşürülür. Kod çözücü bu kodu 12 adım boyunca tekrarlar, ikinci bir LSTM (32 birim) ve doğrusal bir katmandan geçirerek öznitelik boyutuna geri döndürür. Model 18 öznitelik için 16 250 (test 1'in 36 özniteligi için 19 148) parametreye sahiptir. Model Adam ile (öğrenme oranı 2×10^{-3} , yığın boyutu 64) 60 epok boyunca *gürültü giderici* (denoising) otokodlayıcı olarak eğitilir: girdiye standart sapması 0,1 olan Gauss gürültüsü eklenir ve modelden temiz pencereyi yeniden oluşturması istenir. Yalnızca yaklaşık 1 000 pencerelek bir eğitim kümesinde (test 2) bu, ağır eğitim verisini

ezberlemesini önler. Pencereleer rulman bazında oluşturulur, hiçbir zaman iki rulmanı kapsamaz ve t kaydının penceresi t 'de biter; dolayısıyla puan yalnızca geçmiş veriyi kullanır ve çevrimiçi (online) çalışabilir.

Kod 6: LSTM otokodlayıcı (src/bearing_efw/models.py'den alıntı).

```

1 class Net(nn.Module):
2     def __init__(self):
3         super().__init__()
4         self.enc = nn.LSTM(n_features, hidden, batch_first=True)
5         self.to_latent = nn.Linear(hidden, latent)
6         self.from_latent = nn.Linear(latent, hidden)
7         self.dec = nn.LSTM(hidden, hidden, batch_first=True)
8         self.out = nn.Linear(hidden, n_features)
9
10    def forward(self, x):
11        # x: (batch, window, features)
12        _, (h, _) = self.enc(x)
13        z = self.to_latent(h[-1])
14        # latent code
15        rep = self.from_latent(z).unsqueeze(1).repeat(1, x.shape[1], 1)
16        y, _ = self.dec(rep)
17        return self.out(y)
18
19    def fit(self, Xs):
20        # Windows are built per bearing so a sequence never spans two bearings.
21        W = torch.tensor(np.concatenate([self._windows(X)[self.window - 1:] for X in Xs]))
22        opt = torch.optim.Adam(self.net.parameters(), lr=self.lr)
23        for _ in range(self.epochs):
24            for idx in np.array_split(self.rng.permutation(len(W)), max(1, len(W) //
25            self.batch_size)):
26                xb = W[idx]
27                # denoising objective: reconstruct the clean window from a noisy copy
28                loss = ((self.net(xb + self.noise * torch.randn_like(xb)) - xb) ** 2).mean()
29                opt.zero_grad(); loss.backward(); opt.step()
30
31    def score(self, X):
32        with torch.no_grad():
33            W = torch.tensor(self._windows(X))
34            # window ending at each snapshot
35            return ((self.net(W) - W) ** 2).mean(dim=(1, 2)).numpy()

```

5.5 Puandan alarm

Eşik. Her rulman ve dedektör için eşik yalnızca kalibrasyon puanlarından (s) türetilir:

$$\tau = Q_{0.995}(s) + m(Q_{0.995}(s) - \text{medyan}(s)), \quad m = 0,5. \quad (8)$$

Pay (margin), sağlıklı puanların yayılımına göre ifade edildiğinden kural her ölçek ve işaretteki puan için çalışır (RMS z-puanı negatif olabilir, diğer puanlar olamaz).

Debounce. Eşiği aşan tek bir gürültülü kayıt operatörü uyarmamalıdır. Son $n = 6$ kaydın (bir saat) en az $k = 4$ 'ü τ 'yu aştığında t anında alarm etkindir.

Kalıcı alarm. Gerçek bir bozulma kendiliğinden düzelmez. Arızalı bir rulman için raporlanan uyarı, kaydın sonuna kadar sürmesi şartıyla *son* alarm bölümünün başlangıcıdır; daha önce başlayıp sona eren bölümler ayrıca sayılır.

Kod 7: Eşik, k-of-n debounce ve kalıcı alarm (src/bearing_efw/alarm.py).

```

1 def fit_threshold(calib_scores, quantile, margin):
2     q, med = np.quantile(calib_scores, quantile), np.median(calib_scores)

```

```

3     return float(q + margin * (q - med))
4
5 def k_of_n(exceed: np.ndarray, k: int, n: int) -> np.ndarray:
6     """Alarm at t when at least k of the last n snapshots (inclusive) exceed the threshold."""
7     c = np.convolve(exceed.astype(int), np.ones(n, dtype=int))[: len(exceed)]
8     return c >= k
9
10 def first_persistent_alarm(alarm, hours, start_h):
11     """Start of the final alarm episode, if it lasts until the end of the record."""
12     eps = [(s, e) for s, e in alarm_episodes(alarm) if hours[s] >= start_h]
13     if not eps:
14         return None
15     return float(hours[eps[-1][0]]) if eps[-1][1] == len(alarm) else None

```

5.6 Teşhis

Kalıcı bir alarm verildiğinde sistem, alarm veren rulmanın standartlaştırılmış zarf özniteliklerine sonraki 12 kayıt boyunca bakar ve ortalama z-puanı en yüksek olan arıza frekansını, 3'ü aşıyorsa, raporlar (Kod 8). Aksi hâlde tahmin yürütmek yerine “belirgin arıza frekansı yok” der.

Kod 8: Zarfa dayalı teşhis (scripts/run_experiment.py’den alıntı).

```

1 DEFECTS = {"env_bpfo": "outer race", "env_bpfi": "inner race", "env_bsf": "rolling element"}
2
3 def diagnose(d, alarm_h, n=12):
4     if alarm_h is None:
5         return ""
6     cols = list(d.raw.columns)
7     after = np.flatnonzero(d.hours >= alarm_h)[:n]
8     z = {k: np.mean([d.X[after, cols.index(c)].mean() for c in cols if c.startswith(k + "_")])
9           for k in DEFECTS}
10    best = max(z, key=z.get)
11    return f"{DEFECTS[best]} (z={z[best]:.1f})" if z[best] > 3 else "no clear defect frequency"

```

5.7 Değerlendirme protokolü

Veri seti yalnızca her test durdurulduğunda bir rulmanın arızalı olduğunu belirtir; bozulmanın başlangıcı etiketli değildir. Buradan iki metrik çıkar:

- Arızalı bir rulmanın **uyarı süresi**: kalıcı alarmı ile testin bitişi arasındaki saat (test süresinin yüzdesi olarak da verilir). Yüksek olması daha iyidir.
- **Yanlış alarmlar**: sağlıklı olduğuna inanılan bir pencere içinde *herhangi bir* rulmanda verilen alarm bölümleri. Bu pencere kalibrasyondan sonra (kayıtların %35’i) başlar ve test 1, 2 ve 3 için sırasıyla kayıtların %60, %50 ve %85’inde biter. Bu sınırlar durum göstergelerinin incelenmesinden gelir (Bölüm 6) ve yapılandırma dosyasında belgelenmiştir. Oran, bu penceredeki 1 000 rulman-saat başına normalleştirilir.

Hiperparametreler (bölme oranları, eşik yüzdeliği ve payı, k ve n , ağ boyutu, epok sayısı, gürültü) yalnızca test 2 üzerinde seçilmiştir. Test 1 ve test 3 hiçbir karar için kullanılmamış ve ayrılmış değerlendirme kümeleri olarak kalmıştır.

5.8 Tasarım kararları

- **Baştan eğit, gerisinde test et.** Tüm bölmeler zaman sıralıdır. Tüm deneyden alınan pencereleri karıştırmak arızayı eğitime sızdırır.

- **Bir kayıt, zamanda bir noktadır.** Öznitelikler dosya başına hesaplanır, böylece sonuçlar “arızadan kaç saat önce” cinsinden ifade edilir.
- **Doğru rulmanı ve her rulmanı izle.** Her rulman ayrı puanlanır; arızalanan rulmanlar veri setinin belgelerinden bilinir ve yalnızca değerlendirmede kullanılır.
- **Saate göre değil, kayıt sayısına göre böl.** Test 1’deki çok günlük boşluklar nedeniyle *sürenin* ilk %25’i çok az kayıt içerirdi.
- **Rulman bazında temel seviye.** Her rulmanın kendine göre standartlaştırılması, doğal titreşim seviyeleri farklı rulmanların tek bir modelle izlenmesini sağlar.
- **Tüm dedektörler için aynı eşik kuralı ve alarm mantığı;** böylece sonuçlardaki farklar ayar farkından değil dedektörlerin kendisinden gelir.
- **Belirlenimcilik (determinism).** Tüm rastgele bileşenler sabit tohumla başlatılır; art arda iki çalıştırma aynı sonuç tablolarını üretir.

6 Sonuçlar

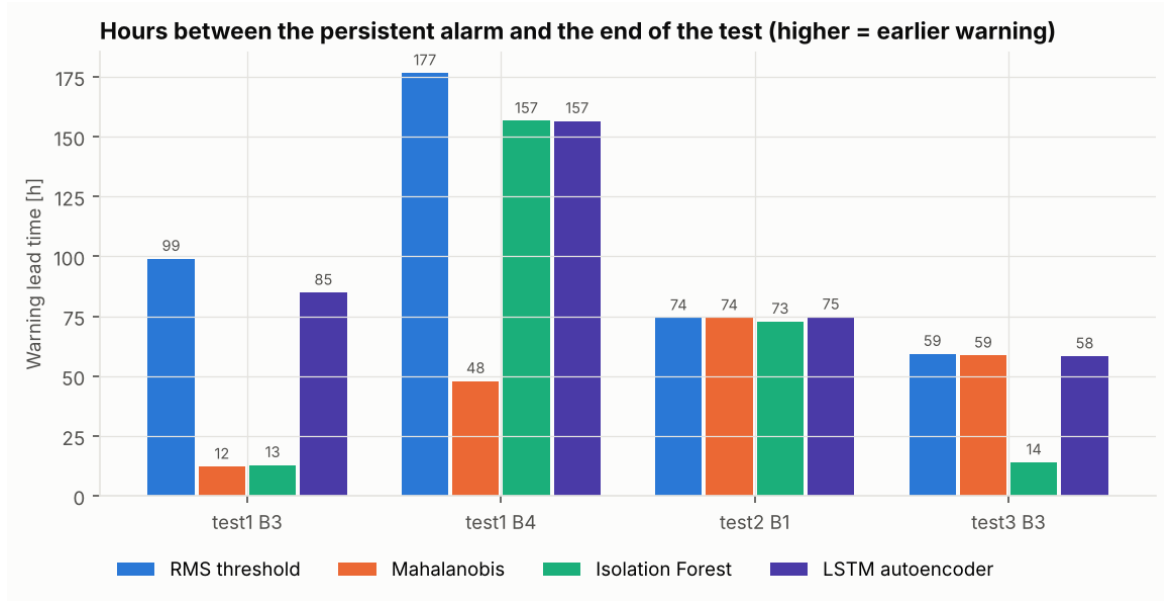
6.1 Ana sonuçlar

Tablo 5: Her arızalı rulmanda kalıcı alarmın uyarı süresi (testin bitişinden kaç saat önce) ve yanlış alarmlar. Kalın: sütundaki en iyi değer.

Dedektör	test1 (ayrılmış)		test2 (gelişt.)	test3 (ayrılmış)	Yanlış alarm / 1000 rulman-sa
	R3 iç bil.	R4 yuv. el.	R1 dış bil.	R3 dış bil.	
RMS eşiği	99,0	176,6	74,3	59,3	5,34
Mahalanobis	12,2	48,0	74,3	58,7	0,67
Isolation Forest	12,9	156,6	72,8	14,0	0,00
LSTM otokodlayıcı	84,9	156,6	74,7	58,3	12,67

Tablo 6: Test başına yanlış alarm bölümleri ve değerlendirme penceresinin boyutu.

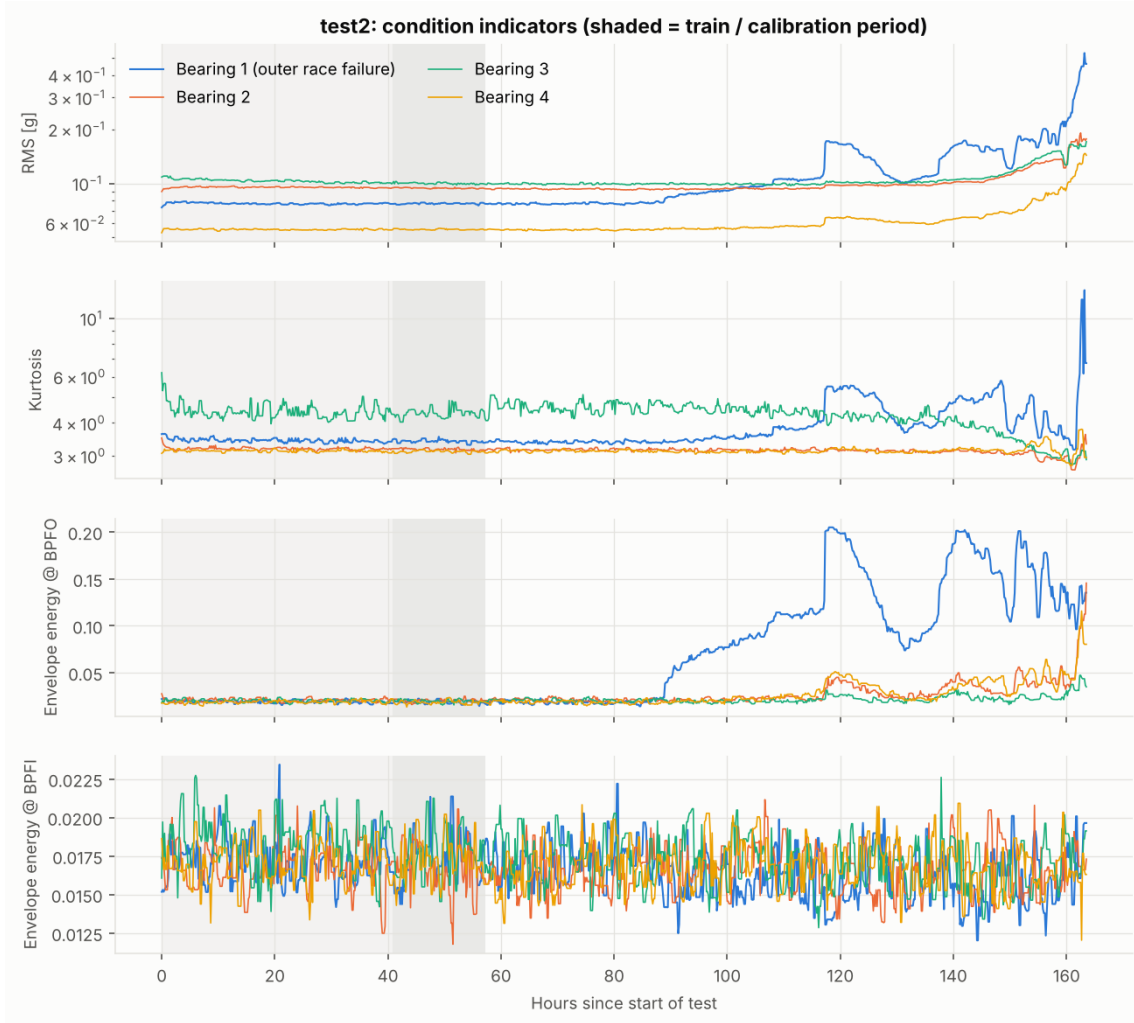
Dedektör	test1	test2	test3
RMS eşiği	15	0	1
Mahalanobis	0	0	2
Isolation Forest	0	0	0
LSTM otokodlayıcı	4	2	32
Sağlıklı pencere (saat)	434–632	57–82	380–908
Sağlıklı pencere (rulman-saat)	3 162	389	8 443



Şekil 5: Her arızalı rulman ve dedektör için kalıcı alarmin uyarı süresi.

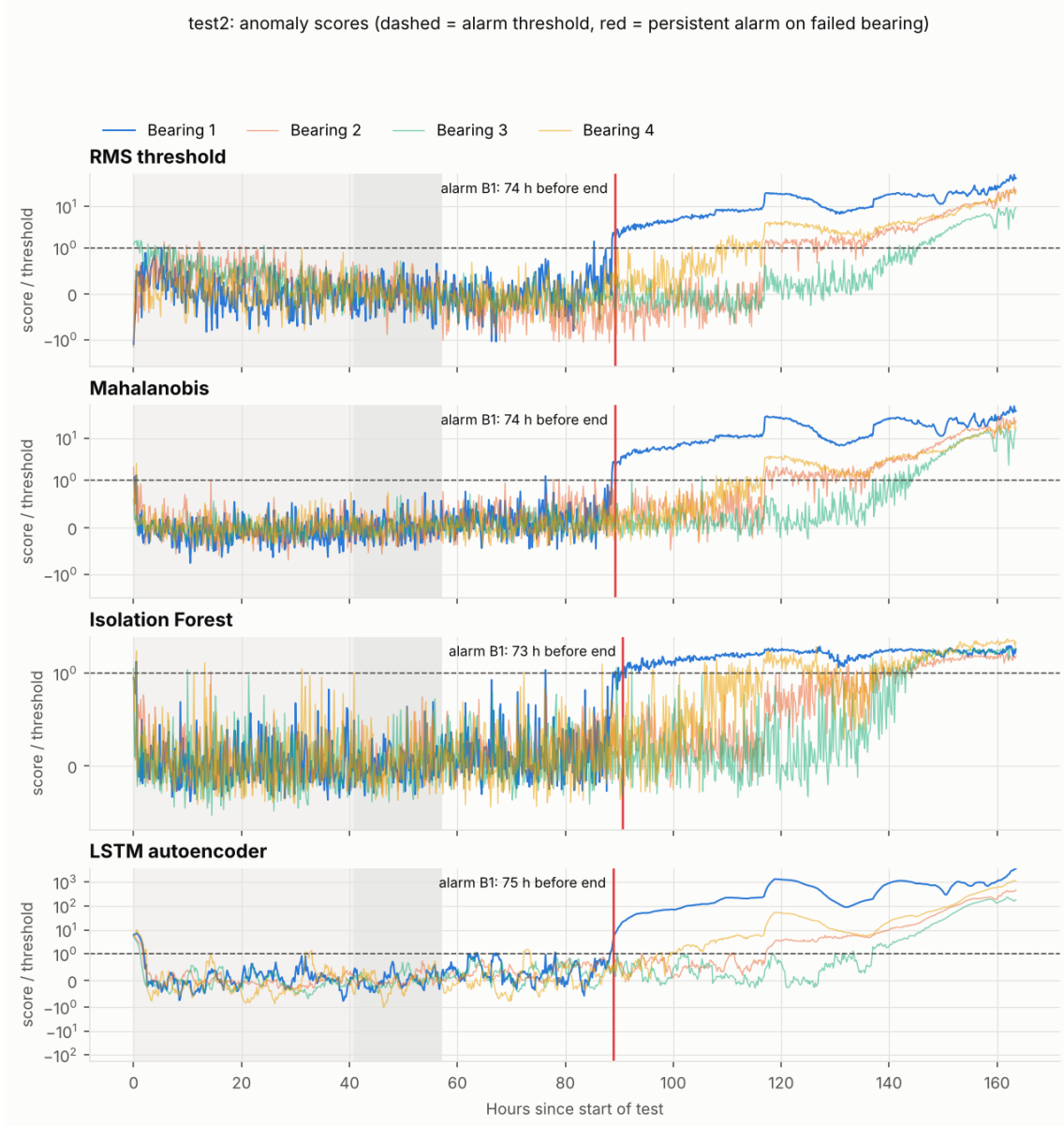
Şekil 5'in yorumu. Test 2'de dört sütun pratikte eşittir (73–75 sa): bir hasar temiz biçimde geliştiğinde makul her dedektör onu aynı anda görür ve dedektör seçimi önemli değildir. Ayrılmış testler dedektörleri birbirinden ayırır. Test 1'de RMS temel modeli ve LSTM otokodlayıcı en erken uyarıyı verir (85–177 sa); Mahalanobis uzaklığı ise bitişten yalnızca 12–48 sa önce uyarır. Test 3'te üç dedektör yaklaşık 59 sa, Isolation Forest ise yalnızca 14 sa verir. Tablo 6 ile birlikte okunduğunda, en erken uyarı veren dedektörlerin aynı zamanda en çok yanlış alarm üreten dedektörler olduğu görülür; bedava öğle yemeği yoktur.

6.2 Test 2 (geliştirme kümesi)



Şekil 6: Test 2'nin durum göstergeleri (5 kayıtlık kayan medyan). Gölge alanlar: eğitim ve kalibrasyon dönemleri.

Yorum. İlk 85 saat boyunca dört rulmanın tüm göstergeleri düzdür. Yaklaşık 88. saatte rulman 1'in BPFO zarf enerjisi 0,02'den 0,05'e sıçrar ve 118. saatte yaklaşık 0,2'ye kadar yükselmeye devam eder; RMS ise yalnızca yavaşça artar. Bu, Şekil 4'de görülen dış bilezik hasarıdır. Rulman 1'in basıklığı da (darbeler) artar, ancak daha kademeli olarak. BPFI zarf enerjisi tüm rulmanlar için gürültü seviyesinde kalır; bu, iç bilezik değil dış bilezik hasarıyla tutarlıdır. Yaklaşık 115. saatten itibaren diğer rulmanların RMS'i ve BPFO enerjisi de artar: rulman 1'in titreşimi mil ve gövde üzerinden diğerlerine iletilir. Son saatlerde tüm göstergeler dik biçimde yükselir. Rulman 1'in BPFO enerjisinin 130. saat civarındaki "çukuru" oyuk (spall) gelişiminde tipiktir: hasarın kenarları aşınıp yumuşadıkça darbeler geçici olarak zayıflar, ardından hasar yayılır.



Şekil 7: Test 2 anomali puanlarının her rulmanın eşiğine oranı (simetrik log ölçek). Kesikli çizgi: alarm eşiği; kırmızı çizgi: arızalı rulmandaki kalıcı alarm.

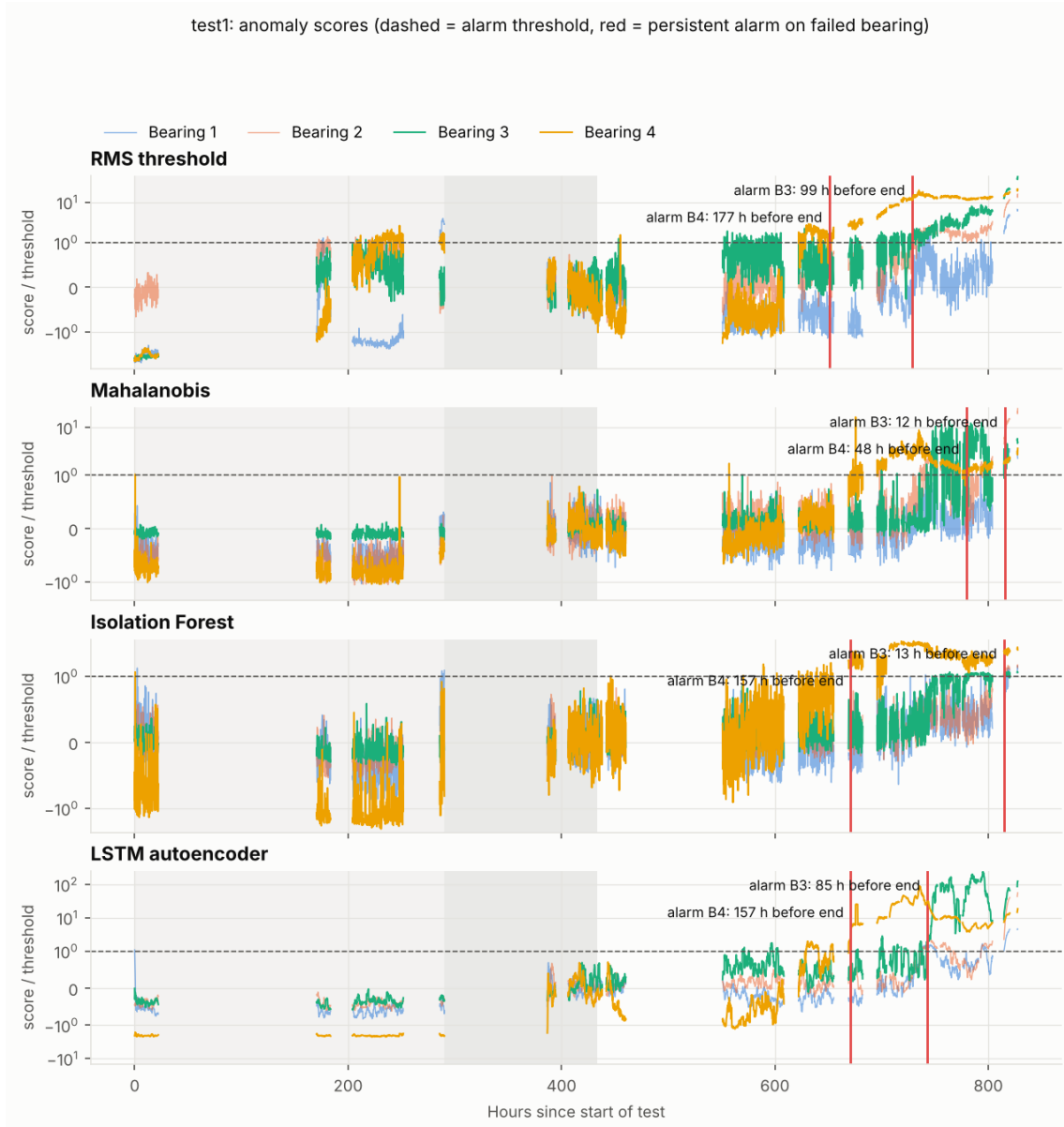
Yorum. Dört dedektör de eğitim, kalibrasyon ve sonraki 30 saat boyunca tüm rulmanları eşik altında tutar; ardından rulman 1 89. saatte eşik aşar ve bir daha altına inmez: bitişten 74 saat önce kalıcı alarm. Puan sıçraması tüm öznelikleri birleştiren LSTM otokodlayıcıda (iki mertebe) ve Mahalanobis uzaklığında (yaklaşık 20×) en büyük, puanı yapısı gereği doygunluğa ulaşan Isolation Forest'ta en küçüktür. İki kısa LSTM bölümü (yalnış alarm olarak sayılır) dışında diğer rulmanlar eşik ancak 109–145. saatlerden sonra ve her zaman rulman 1'den sonra aşar; dolayısıyla *ilk alarm veren rulman* kaynağı doğru biçimde gösterir. Tüm dedektörler alarm anında **dış bilezik** hasarı teşhis eder (z-puanları 6,1–7,3); bu, belgelenmiş arıza tipidir. Rulman 2 ve 4 de sonraki alarmlarında dış bilezik teşhisi alır, çünkü rulman 1'in titreşimini taşırlar; panelin ilk alarm veren rulmanı muhtemel kaynak olarak işaretlemesinin nedeni budur.

6.3 Test 1 (ayrılmış)



Şekil 8: Test 1'in durum göstergeleri. Çizgi kesintileri kayıt boşluklarıdır.

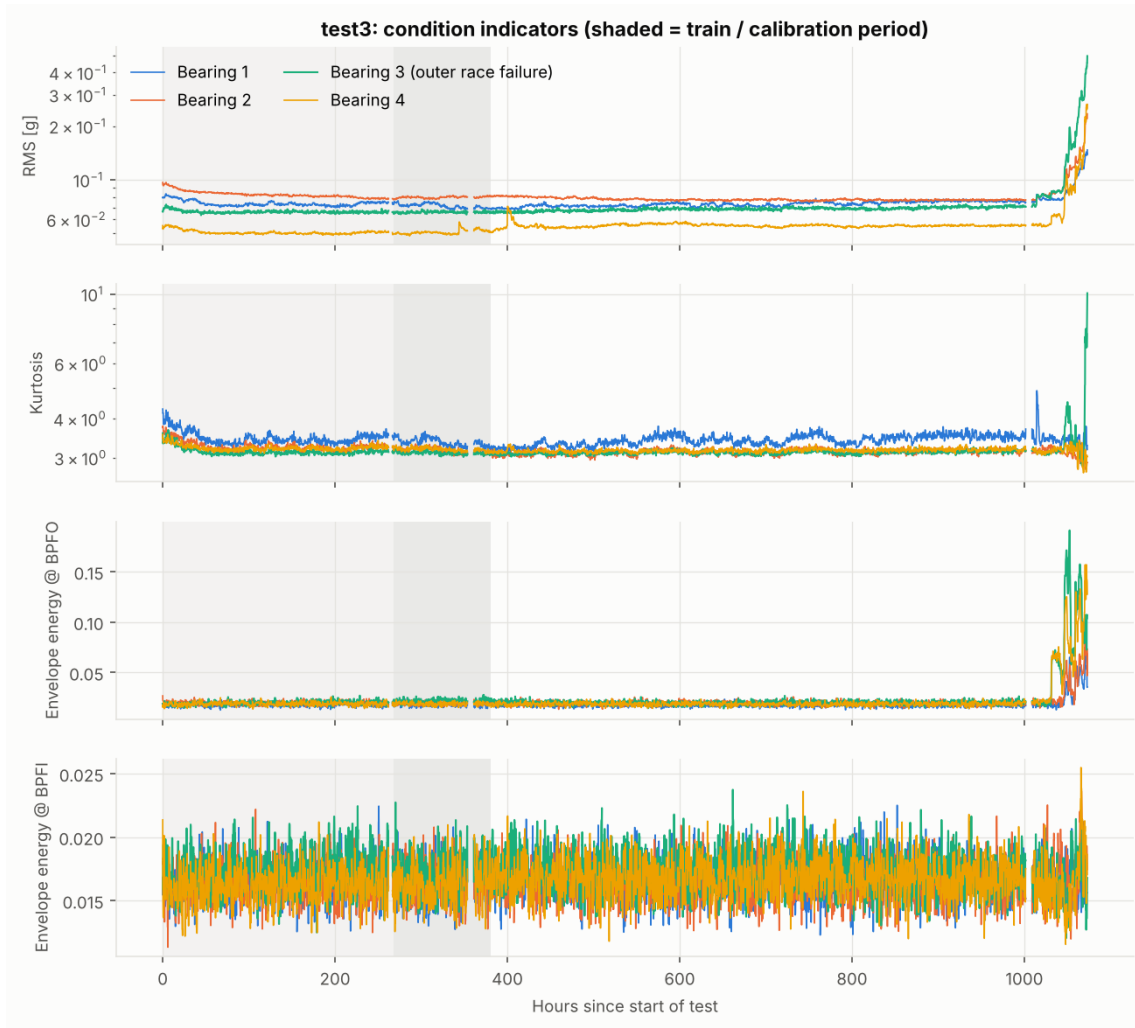
Yorum. Test 1 en zor kayıttır: altı güne varan boşluklarla parçalanmıştır ve eğitim dönemi (ilk 539 kayıt, 290. saate kadar) kayıt parçaları arasında birkaç basamak değişimi içerir; yani “sağlıklı” tek ve kararlı bir durum değildir. İki arızalı rulmanın bozulması basıklıkta açıkça görülür: rulman 4 yaklaşık 680. saatten itibaren (basıklık 20'ye kadar), rulman 3 ise yaklaşık 720. saatten itibaren (40'a kadar) darbeleri hâle gelir; bu, RMS'leri artmadan çok öncedir. BPFI'deki zarf enerjisi (rulman 3'ün iç bilezik hasarı) hiç artmaz, rulman 4'ün BPFO enerjisi de yalnızca dalgalanır. İç bilezik ve yuvarlanma elemanı hasarları yük bölgesine girip çıktıkları için zarf enerjilerini tek bir çizgide toplamak yerine $\pm f_r$ ve kafes frekansındaki yan bantlara yayarlar; harmoniklerin çevresindeki ± 3 Hz'lik pencere bu enerjinin çoğunu kaçıır.



Şekil 9: Test 1 anomali puanlarının her rulmanın eşğine oranı.

Yorum. Rulman 4, üç dedektör tarafından bitişten 157–177 saat önce (deneyin %19–21’i) tespit edilir ve her dedektör için ilk alarm veren rulmanlar iki arızalı rulmandır; yani kaynak tespiti doğrudur. RMS temel modeli en erken uyarıyı verir, ancak sağlıklı kabul edilen pencere içinde (434–632. saatler) eşği etrafında salınarak rulman 3’te (554. saatten itibaren) ve rulman 4’te (623. saatten itibaren) toplam 15 alarm bölümü üretir. Bunlar tam da daha sonra arızalanan iki rulman olduğu için bu bölümlerin bir kısmı aslında yanlış alarm değil, bozulmanın ilk işaretleri olabilir; etiketli bir başlangıç olmadığından değerlendirme bunları temkinli biçimde yanlış alarm sayar. Mahalanobis uzaklığı ve Isolation Forest hiç yanlış alarm vermez, ancak rulman 3’teki kalıcı alarmları bitişten yalnızca 12–13 sa önce başlar: aslında eşği çok daha önce aşmışlardır (rulman 3 için ilk alarm 747. ve 756. saatlerde), fakat puan yeniden eşğin altına indiği için temkinli “kalıcı alarm” tanımı bu uyarıları dışarıda bırakır. Bu testte en iyi denge LSTM otokodlayıcıdadır (85 sa ve 157 sa, 4 yanlış alarm bölümü). Yukarıda açıklanan nedenle hiçbir dedektör test 1’de güvenilir bir arıza tipi teşhisi üretmez.

6.4 Test 3 (ayrılmış)



Şekil 10: Test 3'ün durum göstergeleri.

Yorum. Test 3 1 000 saat boyunca neredeyse hiç bozulma belirtisi göstermez. İki olay önemlidir. Birincisi, 354. saat civarındaki 7,5 saatlik duruştan sonra tüm rulmanların RMS'i hafifçe kayar ve rulman 4 yaklaşık 400. saatte kısa bir sıçrama gösterir: makine biraz farklı bir koşulda yeniden çalışmaya başlamıştır. İkincisi, 1 002. saatteki 7,3 saatlik duruştan sonra tüm göstergeler değişir ve yaklaşık 1 040. saatten itibaren rulman 3'ün (ve iletim yoluyla rulman 4'ün) BPFO zarf enerjisi keskin biçimde yükselir; son 20 saatte bunu RMS ve basıklık izler. Dolayısıyla rulman 3'teki dış bilezik hasarı 44 günlük bir deneyin yalnızca son %5'inde görünür hâle gelir.



Şekil 11: Test 3 anomali puanlarının her rulmanın eşğine oranı.

Yorum. RMS, Mahalanobis ve LSTM dedektörleri rulman 3'te bitişten yaklaşık 59 sa önce kalıcı alarm verir; ancak bu alarm duruştan sonra kaydın yeniden başlamasından (1009. saat) yaklaşık 5 saat sonra başlar ve rulman 1'deki alarmlarla çakışır. Dolayısıyla bu alarmın bir kısmı hasara değil yeniden başlatmaya verilen tepkidir ve bu dedektörler hangi rulmanın arızalandığını ayırt edemez (rulman 1 ve 3 aynı kayıta alarm verir). Alarmı açıkça hasardan kaynaklanan tek dedektör Isolation Forest'tır: BPFO enerjisi sıçradığında, bitişten 14 sa önce alarm verir ve 11,4 z-puanıyla **dış bilezik** hasarı teşhis eder. 354. saatten sonraki rejim değişimi yanlış alarmların ana kaynağıdır: tüm özneteliklerin ortak değişimini modelleyen LSTM otokodlayıcı yeni temel seviyeyi anormal olarak yorumlar ve çoğu rulman 1'de olmak üzere 32 bölüm üretir. Bu, deneyin başında bir kez eğitilen bir modelin en önemli sınırlamasıdır.

6.5 Bulguların özeti

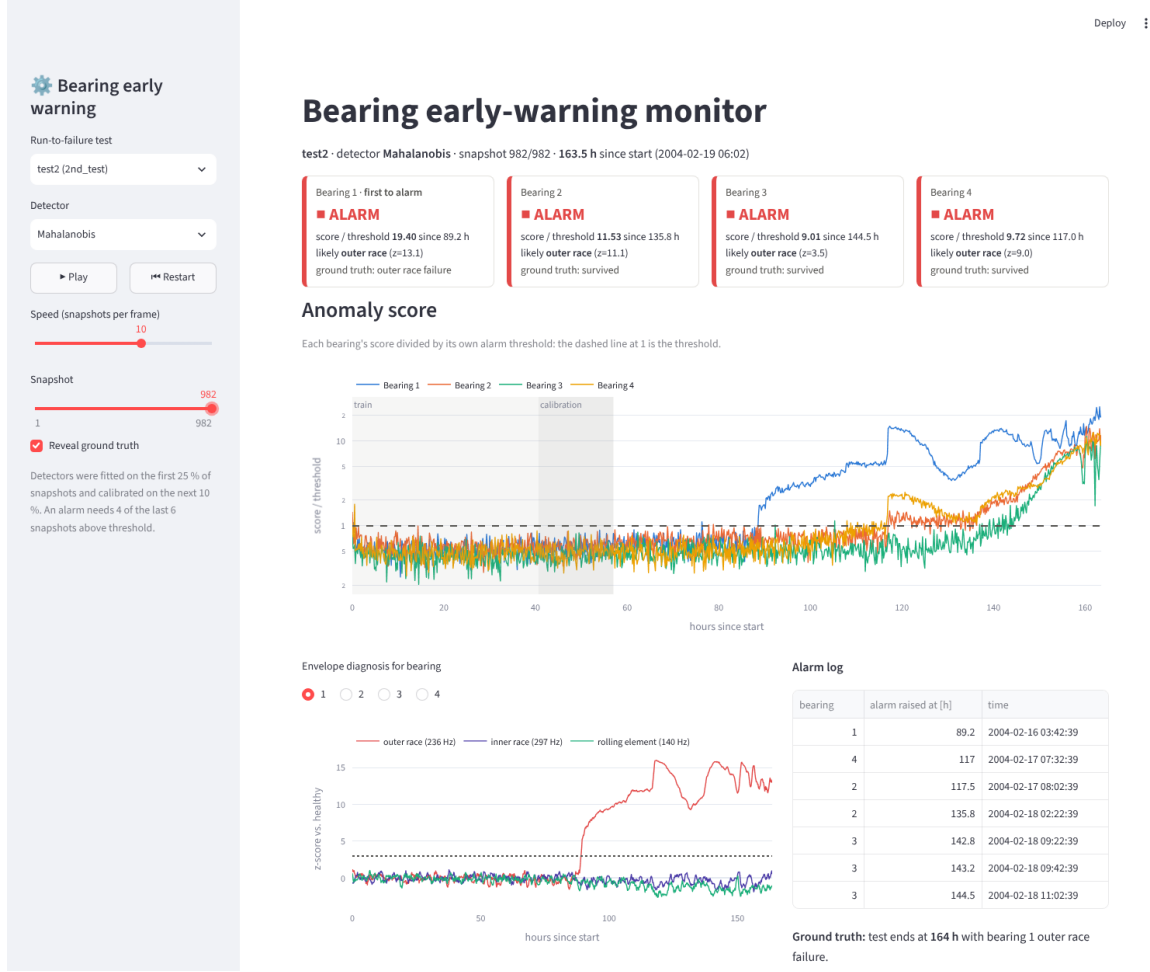
1. **Hasar temiz biçimde geliştiğinde erken uyarı çalışır.** Test 2'de tüm dedektörler bitişten 74 sa (deneyin %45'i) önce uyarır ve doğru arıza tipini belirtir.

2. **Uyarı süresi ile yanlış alarm arasında ödünleşim vardır.** RMS temel modeli ve LSTM otokodlayıcı en erken uyarır; Isolation Forest hiç yanlış alarm vermemiş ama dört arızanın ikisinde geç uyarmıştır; Mahalanobis uzaklığı çok az yanlış alarmla arada yer alır.
3. **Güçlü bir temel model önemlidir.** Basit RMS eşiği uyarı süresinde rekabetçidir. Yalnızca üç deney içeren bir veri setinde derin model üstünlük kurmaz.
4. **İlk alarmla kaynak tespiti** test 1 ve test 2’de güvenilirdir, ancak test 3’teki yeniden başlatmadan sonra değildir.
5. **Zarf teşhisi dış bilezik hasarları için güvenilirdir** (test 2 ve test 3), ancak mevcut dar bantlı öznelikle test 1’in iç bilezik ve yuvarlanma elemanı hasarlarını tanımaz.
6. **Çalışma koşulu değişiklikleri** (yeniden başlatma, bakım) geçerliliğe yönelik ana tehdittir: ömrün başında bir kez eğitilen bir model yeni bir normal anomali olarak görür.

7 Etkileşimli Panel (Dashboard)

Sistemin bir operatöre nasıl görüneceğini göstermek için `app/dashboard.py` herhangi bir testi makineden canlı veri geliyormuş gibi yeniden oynatır (Şekil 12). Kullanıcı bir test ve bir dedektör seçip *Play*’e basar; panel kayıt kayıt ilerler ve şunları gösterir:

- her rulman için renk, simge ve etiketle bir durum kartı (sağlıklı / eşik üstünde / alarm), puan/eşik oranı, mevcut alarmın başlangıcından bu yana geçen süre, muhtemel kaynak için *ilk alarm veren* (first to alarm) işareti ve teşhis edilen arıza tipi;
- eğitim ve kalibrasyon dönemleri gölgelenmiş, o ana kadarki tüm rulmanların anomali puanı grafiği;
- seçilen rulman için BPFO, BPFİ ve BSF’deki zarf z-puanları;
- bir alarm günlüğü ve istenirse gerçek durum (hangi rulmanın nasıl arızalandığı).



Şekil 12: Test 2'nin sonunda, Mahalanobis dedektörüyle ve gerçek durum gösterilirken panel.

Yorum. Kartlar test 2'nin hikâyesini tek satırda özetler: rulman 1 belirgin bir dış bilezik imzasıyla ($z = 13,1$) ilk alarm veren rulmandır (89,2. saatten beri); diğerleri 28–55 saat sonra onu izler. Zarf grafiği, rulman 1'in BPFO z-puanının 90. saat civarında 0'dan 10'un üstüne sıçradığını, BPFİ ve BSF'nin ise düz kaldığını gösterir: teşhisin arkasındaki kanıt operatör tarafından görülebilir.

Kod 9: Muhtemel kaynak ve her rulmanın durumu (app/dashboard.py'den alıntı).

```

1 # The first bearing to raise an alarm is the likely source: once it degrades, its
2 # vibration travels through the shaft and housing and lifts its neighbours too.
3 onsets = {b: g.loc[g["alarm"], "hours"].min() for b, g in seen.groupby("bearing") if
4           g["alarm"].any()}
5 source = min(onsets, key=onsets.get) if onsets else None
6
7 for col, (b, g) in zip(cols, seen.groupby("bearing")):
8     last = g.iloc[-1]
9     state = "alarm" if last.alarm else ("watch" if last.score > last.threshold else "ok")
10    defect, zval = diagnose(zs[b].loc[:now]) # mean z of the last 6 snapshots
11    ...
12
13 # Playback: advance the slider and rerun the script
14 if st.session_state.playing and idx < n:
15     st.session_state["advance_to"] = min(n, idx + speed)
16     time.sleep(0.15)
17     st.rerun()

```

Panel yalnızca depoya eklenmiş sonuç dosyalarını okur, bu nedenle ham veri olmadan çalışır: make dashboard komutu paneli <http://localhost:8501> adresinde açar.

8 Yazılım Mühendisliği ve Yeniden Üretilebilirlik

8.1 Depo yapısı

1	configs/default.yaml	splits, thresholds, alarm rule, model hyper-parameters
2	src/bearing_efw/	
3	dataset.py	test metadata, bearing geometry, defect frequencies, loader
4	features.py	time, spectral and envelope features per snapshot
5	preprocess.py	per-bearing matrices, log + standardisation, splits
6	models.py	RMS, Mahalanobis, Isolation Forest, LSTM autoencoder
7	alarm.py	thresholding, k-of-n debouncing, alarm episodes
8	plots.py	report figures
9	scripts/	download_data, build_features, run_experiment, make_report_figures
10	app/dashboard.py	Streamlit replay dashboard
11	data/features/	extracted features (committed, 3 MB)
12	reports/	results.md, summary.csv, scores.csv.gz, figures/
13	notebooks/	01_walkthrough.ipynb
14	tests/	unit tests and dashboard smoke tests
15	report/	this report (LaTeX sources and PDFs, English and Turkish)

8.2 Yapılandırma

Deneydeki tüm seçimler tek bir YAML dosyasında tutulur; böylece deney kod değiştirmeden değiştirilebilir:

```

1 split:      {train_end: 0.25, calib_end: 0.35}
2 healthy_end: {test1: 0.60, test2: 0.50, test3: 0.85}
3 threshold:  {quantile: 0.995, margin: 0.5}
4 alarm:      {k: 4, n: 6}
5 models:
6   isolation_forest: {n_estimators: 300, seed: 0}
7   lstm_ae: {window: 12, hidden: 32, latent: 8, epochs: 60, lr: 0.002,
8             batch_size: 64, noise: 0.1, seed: 0}

```

8.3 Çalıştırma

```

1 pip install -r requirements.txt
2 make experiment # ~2 min, uses the committed feature tables
3 make dashboard # interactive replay at http://localhost:8501
4 make test
5 make data      # optional: download + extract raw data (~1 GB / 6 GB)
6 make features  # optional: rebuild the feature tables from raw data

```

8.4 Testler

On iki otomatik test, sessizce bozulması en olası kısımları denetler. En önemlisi, gürültü içine gömülmüş ve tam olarak BPFO hızında tekrarlanan rezonans patlamalarından oluşan yapay bir dış bilezik hasarı üretir ve zarf özneliliğinin bunu tespit edip dış bileziğe atfettiğini kontrol eder (Kod 10). Diğer testler arıza frekanslarını yayımlanmış değerlerle, Gauss gürültüsünün istatistiklerini (RMS = 1, basıklık = 3), debounce ve kalıcılık mantığını, eşğin puan kaymasına karşı değişmezliğini ve panelin her test için iki dedektörle hatasız açıldığını denetler.

Kod 10: Yapay arıza testi (tests/test_core.py'den alıntı).

```

1 def test_envelope_picks_up_outer_race_impacts():
2     """Synthetic outer-race defect: resonance bursts repeating at BPFO."""

```

```

3   rng = np.random.default_rng(1)
4   t = np.arange(FS) / FS
5   bpfo = fault_frequencies()["bpfo"]
6   impacts = np.zeros(FS)
7   impacts[(np.arange(0, 1, 1 / bpfo) * FS).astype(int)] = 1.0
8   ring = np.exp(-t[:200] * 2000) * np.sin(2 * np.pi * 4000 * t[:200])
9   faulty = np.convolve(impacts, ring)[:FS] * 3 + 0.3 * rng.standard_normal(FS)
10  healthy = 0.3 * rng.standard_normal(FS)
11  f_faulty, f_healthy = snapshot_features(faulty), snapshot_features(healthy)
12  assert f_faulty["env_bpfo"] > 5 * f_healthy["env_bpfo"]
13  assert f_faulty["env_bpfo"] > f_faulty["env_bpfi"]
14  assert f_faulty["kurtosis"] > f_healthy["kurtosis"]

```

8.5 Sürüm kontrolü

Proje GitHub’da barındırılmaktadır (https://github.com/ecemaydemir/bearing_early_fault_warning). İlk veri işleme hattı doğrudan main dalına eklenmiş; sonraki eklemeler (panel, bu rapor) ayrı dallarda geliştirilip pull request’lerle birleştirilmiştir.

9 Tartışma ve Sınırlamalar

- **Gerçek etiket (ground truth).** Yalnızca “test sonunda arızalıydı” bilgisi vardır. Bu nedenle uyarı süresi, bilinen bir başlangıcın tespitini değil bir alarmin ne kadar erken kalıcı hâle geldiğini ölçer; yanlış alarm için kullanılan sağlıklı pencereler belgelenmiş birer varsayımdır.
- **Küçük örneklem.** Üç deney ve dört arızalı rulman, istatistiksel olarak sağlam sıralamalar için yeterli değildir; sayılar başka ayarlarla değişecektir. Ayrılmış testlerin sonuçlarını dürüst tutmak için ayarlar yalnızca test 2 üzerinde seçilmiştir.
- **Durağan olmayan çalışma.** Test 3’teki yeniden başlatmalar her rulmanın temel seviyesini kaydırır. Gerçek bir tesiste temel seviye, bakımdan sonra veya bir operatör değişikliğinin zararsız olduğunu onayladığında yeniden kestirilir.
- **Dar bantlı teşhis.** İç bilezik ve yuvarlanma elemanı hasarları mil ve kafes dönüşüyle modüle edilir; yan bantları hesaba katan veya spektral basıklığa dayalı bant seçimi teşhisi daha genel hâle getirir.
- **Kalıcılık tanımı temkinlidir.** Alarmin sona kadar sürmesini şart koşturmak, aralıklı ama gerçek erken uyarıları (test 1, rulman 3) dışarıda bırakır; uygulamada aralıklı bir alarm da operatör için değerli bir bilgidir.

10 Sonuç ve Gelecek Çalışmalar

Proje, arıza etiketi kullanmayan, çalışan ve yeniden üretilebilir bir rulman erken uyarı hattı ortaya koymaktadır. En temiz deneyde üç gün (deneyin %45’i) önceden uyarır ve doğru hasarı belirtir; daha zor deneylerde ise uyarı süresi ile yanlış alarm arasındaki ödünleşimi açık hâle getirir ve nerede, neden başarısız olduğunu belgeler. En değerli sonraki adımlar şunlardır:

1. yeniden başlatma ve bakımdan sonra uyarlanabilir temel seviye kestirimi (test 3);
2. alarmdan sonra kalan faydalı ömür (RUL) kestirimi; örneğin sağlık göstergesine bir bozulma eğilimi uydurarak;
3. iç bilezik ve yuvarlanma elemanı teşhisi için yan bantları hesaba katan zarf öznitelikleri veya spektral basıklık;

4. erken uyarıyı az yanlış alarmla birleştirmek için bir topluluk alarmı (ör. dört dedektörden ikisi);
5. başka arızaya kadar çalıştırma veri setlerinde (ör. FEMTO/PRONOSTIA, XJTU-SY) doğrulama.

Kaynaklar

- [1] J. Lee, H. Qiu, G. Yu, J. Lin, and Rexnord Technical Services. *Bearing Data Set*. IMS, University of Cincinnati. NASA Prognostics Data Repository, NASA Ames Research Center, 2007.
- [2] H. Qiu, J. Lee, J. Lin, and G. Yu. Wavelet filter-based weak signature detection method and its application on rolling element bearing prognostics. *Journal of Sound and Vibration*, 289(4-5):1066–1090, 2006.
- [3] R. B. Randall and J. Antoni. Rolling element bearing diagnostics—A tutorial. *Mechanical Systems and Signal Processing*, 25(2):485–520, 2011.
- [4] F. T. Liu, K. M. Ting, and Z.-H. Zhou. Isolation forest. In *Proc. IEEE International Conference on Data Mining*, pages 413–422, 2008.
- [5] O. Ledoit and M. Wolf. A well-conditioned estimator for large-dimensional covariance matrices. *Journal of Multivariate Analysis*, 88(2):365–411, 2004.
- [6] S. Hochreiter and J. Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.
- [7] P. Malhotra et al. LSTM-based encoder-decoder for multi-sensor anomaly detection. *ICML Anomaly Detection Workshop*, 2016.